LESSON 07 OF 10 · PROMPT ENGINEERING

The 7-Layer Prompt Architecture

From CLAUDE.md to hooks — building a complete prompt engineering stack

NK
Nick Krzemienski
March 2026 · Agentic Development Series

Lesson 07 of 10 · Nick Krzemienski · March 2026 · Agentic Development Series

The 7-Layer Prompt Engineering Stack – Defense-in-Depth for AI Agents

---

Agentic Development: 10 Lessons from 8,481 AI Coding Sessions (Post 7 of 11)

I used to think prompting an AI coding agent was about writing a good initial message. Describe the task clearly, give it some context, hit enter, hope for the best.

After 8,481 sessions I can tell you: the initial prompt is maybe 10% of what determines whether an agent produces reliable code. The other 90% is the invisible system of rules, hooks, skills, and memory that surrounds every interaction – a defense-in-depth architecture where each layer catches failures that slip through the layers above.

I call it the prompt engineering stack. It has 7 layers. Any single layer provides marginal improvement. All 7 together are transformative.

The Problem: AI Agents Cut Corners

Here is the failure mode that cost me the most time across thousands of sessions: an AI agent makes five file edits in rapid succession without verifying any of them compile. By the time you discover the build is broken, the errors have cascaded across three files and the agent's context is polluted with its own mistakes.

Or this one: the agent helpfully commits code that includes a hardcoded API key from an environment variable you pasted during debugging. Now `sk-proj-abc123` is in your git history forever.

Or this: you ask the agent to fix a CSS bug and it decides the real problem is your entire component architecture, refactoring 15 files you did not ask it to touch.

Or this: the agent finishes a feature, reports "everything is working," and when you actually run the application you discover three compilation errors and a runtime crash. The agent never built the code. It never ran it. It read its own output and concluded it was correct.

Every one of these failures happened to me. Some of them happened dozens of times before I built the systems to prevent them. The prompt stack is the result of that scar tissue – layer upon layer of protection, each one born from a specific, expensive mistake.

The fundamental insight is that AI agents are eager to please. They want to report success. Given the choice between "let me verify this actually works" and "I am confident this is correct

based on my analysis," they choose confidence every time. The prompt stack forces verification at every stage, replacing confidence with evidence.

The Stack at a Glance

```
flowchart TD
    L1["Layer 1\nGlobal CLAUDE.md\n(Universal rules &amp; agent catalog)"]
    L2["Layer 2\nProject CLAUDE.md\n(Repo-specific constraints)"]
    L3["Layer 3\n.claude/rules/*.md\n(Domain rules: git, perf, feature-gate)"]
    L4["Layer 4\nHooks\n(PostToolUse auto-build, PreCommit scan)"]
    L5["Layer 5\nSecurity Scan\n(Secrets, hardcoded paths, DB files)"]
    L6["Layer 6\nSkills\n(ios-validation-runner, functional-validation)"]
    L7["Layer 7\nSession Memory\n(MEMORY.md + project-memory.json)"]

    L1 --> L2
    L2 --> L3
    L3 --> L4
    L4 --> L5
    L5 --> L6
    L6 --> L7

    subgraph SessionStart["Session Start"]
        direction LR
        SS["Loads Layers 1-3"]
    end

    subgraph DuringEdit["During Editing"]
        direction LR
        DE["Triggers Layer 4\n(auto-build hook)"]
    end

    subgraph AtCommit["At Commit"]
        direction LR
        AC["Triggers Layer 5\n(security scan)"]
    end

    subgraph Validation["Validation"]
        direction LR
```

```
        VA["Invokes Layers 6-7\n(skills + memory)"]
    end

    SessionStart -.-> L1
    DuringEdit -.-> L4
    AtCommit -.-> L5
    Validation -.-> L6
```

7-layer stack

7-layer stack
Seven layers, four activation points, one goal: make it structurally impossible for an AI agent to cut corners.

The layers divide into three categories by activation pattern:

- **Static context (Layers 1-3):** Loaded once at session start. These shape the agent's understanding before it takes any action.
- **Dynamic enforcement (Layers 4-5):** Triggered by specific events during the session. These catch problems in real time.
- **Workflow and knowledge (Layers 6-7):** Invoked deliberately during validation and updated across sessions. These accumulate institutional knowledge.

Layer 1: Global CLAUDE.md – The Constitution

The global CLAUDE.md lives at `~/.claude/CLAUDE.md` and loads for every Claude Code session regardless of project. This is where you encode standards that apply universally – your agent orchestration strategy, model routing preferences, delegation rules.

Think of it as the constitution. Individual projects can add laws, but they cannot override the constitution.

In my setup, the global file defines agent routing:

```
# From ~/.claude/CLAUDE.md (global)
# Model routing:
# - Haiku: quick lookups, lightweight scans, narrow checks
# - Sonnet: standard implementation, debugging, reviews
# - Opus: architecture, deep analysis, complex refactors
```

This routing is not about cost optimization (though it helps). It is about matching cognitive depth to task requirements. Opus-level reasoning on a simple rename is wasteful. Haiku-level reasoning on a security architecture review is dangerous. The global file makes the choice explicit so neither you nor the agent has to think about it per-session.

The global file also defines delegation rules – when the agent should hand off work to specialized sub-agents rather than attempting everything itself:

```
# From ~/.claude/CLAUDE.md (global)
# Delegation rules:
# - Complex feature requests → planner agent first
# - Code just modified → code-reviewer agent immediately
# - Build failure → build-error-resolver agent
# - Security concerns → security-reviewer agent
```

Without these delegation rules, agents try to do everything themselves. An agent building a feature will also try to review its own code, debug its own errors, and assess its own security posture – all with the same biases and blind spots that produced the original work. Explicit delegation creates separation of concerns at the agent level, the same principle that makes microservices and modular code more reliable than monoliths.

What belongs in Layer 1:

- Model routing preferences that apply to all projects
- Agent orchestration rules (which specialized agents exist, when to invoke them)
- Universal behavioral mandates (the ones you never want overridden)
- Delegation rules (when to hand off to sub-agents)

What does NOT belong in Layer 1:

- Project-specific build commands (that is Layer 2)
- Domain-specific knowledge (that is Layer 3)
- Anything that varies between repositories

The most important rule in my global file, learned after watching agents waste entire sessions writing elaborate test harnesses instead of building the actual feature:

```
# From CLAUDE.md (project-level template)
# claude-prompt-stack/CLAUDE.md

## Functional Validation Mandate

NEVER: write mocks, stubs, test doubles, or fake implementations
when validating features.

ALWAYS: build and run the real system. Validate through actual
user interfaces or real API calls. Capture evidence (screenshots,
logs, curl output) before claiming completion.
```

That single rule – six lines of text – changed the character of every AI session. The agent stops trying to prove correctness through abstraction and starts proving it through demonstration.

Why does this matter? Because an AI agent writing a test for its own code is circular reasoning. The agent that misunderstood the requirement will write a test that validates its misunderstanding. The test passes. The agent reports success. The feature is broken. Functional validation breaks this cycle by requiring the agent to interact with the real running system.

Layer 2: Project CLAUDE.md – The Local Law

Each project gets its own CLAUDE.md in the repository root. This file contains project-specific mandates, build commands, common pitfalls, and working style rules.

The companion repo ships a template:

```
# From CLAUDE.md (project template)
# claude-prompt-stack/CLAUDE.md

## Project Context

This is a **[LANGUAGE/FRAMEWORK] project** for [PROJECT_DESCRIPTION].

**Key Paths:**
- `src/` — Application source code
- `scripts/` — Build and utility scripts
- `.claude/rules/` — AI agent rules (auto-loaded)
- `.claude/hooks/` — Build and security hooks

**Default Port:** [PORT_NUMBER]
**API Prefix:** `[API_PREFIX]`

**Build Commands:**
  [BUILD_COMMAND]
  [RUN_COMMAND]
```

When I fill in these templates for a real project, the agent stops guessing. It knows the port is 9999, not 3000. It knows the API prefix is `/api/v1` and does not double-prefix routes. It knows the build command and does not try `npm run build` on a Swift project.

The template seems generic. The filled-in version is anything but. Here is what my production iOS project's CLAUDE.md contains that the template does not hint at:

```
## Common Pitfalls (from real sessions)

- **Wrong backend binary**: OLD backend at `/Users/nick/ils/ILSBackend/` returns
  raw data. ALWAYS use `/Users/nick/Desktop/ils-ios/`
- **Deep link UUIDs must be LOWERCASE** — uppercase causes failures
- **`import Crypto` vs `import CryptoKit`**: In Vapor, `Crypto` resolves to a
  different SHA256. Use `CryptoKit`
- **DerivedData path**: `~/Library/Developer/Xcode/DerivedData/ILSApp-*/Build/
  Products/`, NOT `ILSApp/build/`
```

Each pitfall in that list cost at least 30 minutes to discover the first time. The DerivedData path issue cost two hours – the agent kept installing an older binary because it was looking in the wrong directory. Encoding these pitfalls in Layer 2 means the agent has the answer before it encounters the problem.

The working style rules address the behavioral anti-patterns I observed across thousands of sessions:

```
# From CLAUDE.md (project template)
# claude-prompt-stack/CLAUDE.md

## Working Style

### Implement Immediately
When asked to fix or implement something, start with implementation
immediately. Do NOT spend the entire session in planning/discovery
mode reading files repeatedly.

### One Change, One Verify
Make a change. Verify it works. Then make the next change. Do not
batch 5 changes and then discover 3 of them broke the build.

### Stay Focused — No Scope Creep
When fixing a specific bug or build error, stay focused on that issue.
Do NOT escalate into full workspace reorganization, architecture
changes, or broad refactoring unless explicitly asked.
```

These rules emerged from specific, painful sessions. The “Implement Immediately” rule came from watching an agent spend 40 minutes reading every file in a 150-file project before making its first edit. The “No Scope Creep” rule came from asking for a one-line CSS fix and getting back a 15-file component refactor. The “One Change, One Verify” rule came from an agent making 8 edits in sequence, where edit 3 introduced a syntax error that corrupted the meaning of edits 4 through 8.

The relationship between Layer 1 and Layer 2: Layer 1 says “always validate with the real system.” Layer 2 says “the real system is built with `swift build` and runs on port 9999.” Layer 1 provides the mandate; Layer 2 provides the specifics needed to follow it.

When these layers conflict (rare but possible), Layer 2 wins for project-specific settings. If Layer 1 says "use Opus for code reviews" but Layer 2 says "use Sonnet for all tasks in this project," the project-specific instruction takes precedence. This is by design – the global file sets defaults that any project can override.

Layer 3: Rules Files – Deep Domain Context

The `.claude/rules/` directory contains markdown files that provide deep domain knowledge about specific aspects of the project. Claude Code auto-loads all files in this directory at session start.

The companion repo ships 9 rules files. Each one addresses a specific domain:

```
.claude/rules/
├── project.md              # Tech stack, directory structure, build commands
├── agents.md               # Specialized agent roles and invocation rules
├── auto-build-hook.md      # How the auto-build system works
├── ci-cd.md                # CI/CD pipeline configuration
├── development-workflow.md # Anti-patterns and correct workflows
├── feature-gate.md         # Feature gating and premium subscription system
├── git-workflow.md         # Commit format, PR process, branch naming
├── performance.md          # Model selection, context management
└── security.md             # Security scanning rules, blocked patterns
```

Why nine separate files instead of one large rules file? Two reasons. First, modularity. When you update the git workflow, you edit `git-workflow.md` without touching the security rules. Second, attention. Claude Code loads each file as distinct context. A 50-line focused file gets more reliable attention than 50 lines buried in a 500-line document.

The Project Quick Reference

The most impactful rules file is `project.md`. It gives the agent a complete mental model of the codebase:

```
# From .claude/rules/project.md
# claude-prompt-stack/.claude/rules/project.md

## Tech Stack
- Language: [LANGUAGE_AND_VERSION]
- Framework: [FRAMEWORK_AND_VERSION]
- Build System: [BUILD_TOOL]
- Package Manager: [PACKAGE_MANAGER]

## Key Constants
| Item            | Value           |
|-----------------|-----------------|
| Default Port    | [PORT]          |
| API Prefix      | [API_PREFIX]    |
| Config Path     | [CONFIG_PATH]   |

## Architecture Notes
- Navigation: [How routing/navigation works]
- State Management: [How state is managed]
- API Layer: [How API calls are structured]
- Error Handling: [Error handling patterns]
```

In my production project, the architecture notes section prevents the most common agent mistakes. “Navigation: `activeScreen` in SidebarRootView.swift controls routing (not `selectedTab`).” Without this note, agents repeatedly try to use `selectedTab` – a property that does not exist – and waste 20 minutes debugging a build error that is actually an understanding error.

The key constants table is equally important. Without it, agents guess at port numbers, API prefixes, and configuration paths. Each guess that is wrong costs a debugging cycle. The table eliminates the guessing entirely.

Agent Orchestration Rules

The `agents.md` file defines specialized agent roles and when to invoke them:

```
# From .claude/rules/agents.md
# claude-prompt-stack/.claude/rules/agents.md
```

```
## Parallel Task Execution

ALWAYS use parallel agent execution for independent operations:

# GOOD: Parallel execution
Launch 3 agents in parallel:
1. Agent 1: Security analysis of auth module
2. Agent 2: Performance review of data layer
3. Agent 3: Build verification across all targets

# BAD: Sequential when tasks are independent
First agent 1... wait... then agent 2... wait... then agent 3
```

Anti-Pattern Catalog

The `development-workflow.md` file encodes the anti-patterns I have seen most frequently:

```
# From .claude/rules/development-workflow.md
# claude-prompt-stack/.claude/rules/development-workflow.md

| Anti-Pattern                  | Do This Instead                  |
|-------------------------------|----------------------------------|
| Planning without executing    | Plan once, then execute          |
| Skipping validation           | Always verify with real system   |
| Batching too many changes     | One change, one verify           |
| Reading files repeatedly      | Read once, track what you know   |
| Scope creep during bug fixes  | Fix the bug, move on             |
```

Each row in that table represents a pattern I observed in at least 10 separate sessions. The anti-pattern table is a compressed form of institutional knowledge – every row is a lesson that cost hours to learn the first time.

Feature Gating Rules

The `feature-gate.md` file prevents a particularly expensive mistake – bypassing or duplicating the feature gating system:

```
# From .claude/rules/feature-gate.md

## Rules

- NEVER add `if isPremium` checks directly in views — use `FeatureGateView`
- NEVER create new feature types without adding to `FeatureGate.Feature` enum
- NEVER hardcode premium status as `true` even in dev — test both states
- `SubscriptionManager.shared` owns the source of truth for premium status
```

Without this rules file, every agent that touches a premium feature reinvents its own gating mechanism. Some use `if isPremium`. Some query the subscription manager directly. Some hardcode `true` for testing and forget to remove it. The rules file standardizes the approach so every agent uses the same pattern.

Security Rules

The `security.md` file documents the patterns that the pre-commit hook scans for:

```
# From .claude/rules/security.md
# claude-prompt-stack/.claude/rules/security.md

## Blocked Patterns (commit will be rejected)

| Pattern | Risk | Example |
|---------|------|---------|
| `sk-[a-zA-Z0-9]` | API keys (OpenAI, Anthropic) | `sk-proj-abc123...` |
| `AKIA[A-Z0-9]` | AWS access keys | `AKIAIOSFODNN7EXAMPLE` |
| `ghp_[a-zA-Z0-9]` | GitHub personal access tokens | `ghp_xxxxxxxxxxxx` |
| `glpat-[a-zA-Z0-9]` | GitLab personal access tokens | `glpat-xxxxxxxxx` |
| `.sqlite` / `.db` files | Database files with user data | `app.sqlite` |
| `.env` files | Environment config with secrets | `.env.local` |
```

This rules file serves two purposes. First, it tells the agent what the hook will block, so the agent avoids introducing those patterns in the first place. Second, it provides documentation for humans who want to understand the security scanning behavior. Defense in depth means the agent is warned at context-load time (Layer 3) and blocked at commit time (Layer 5).

Layer 4: Auto-Build Hook – The Guardrail That Changed Everything

This is the layer that had the single biggest impact on code quality. The auto-build hook fires automatically after every source file edit and runs the appropriate build command based on file type.

Here is the hook configuration that wires it up:

```
// From .claude/settings.local.json
// claude-prompt-stack/.claude/settings.local.json

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "command": "bash .claude/hooks/auto-build-check.sh \"$CLAUDE_FILE_PATH\"",
        "timeout": 120000
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "command": "if echo \"$CLAUDE_TOOL_INPUT\" | grep -q 'git commit'; then ba
        "timeout": 30000
      }
    ]
  }
}
```

Two hook types. `PostToolUse` fires after every Edit, Write, or MultiEdit operation – meaning every source file change triggers a build. `PreToolUse` fires before the Bash tool runs, and checks if the command is a `git commit` – if so, it runs the security scan.

The `matcher` field uses regex-style matching against the tool name. `Edit|Write|MultiEdit` matches any file editing operation. The `$CLAUDE_FILE_PATH` variable is injected by Claude

Code and contains the path to the file that was just modified.

The routing logic detects file type and runs the right build:

```
# From .claude/hooks/auto-build-check.sh
# claude-prompt-stack/.claude/hooks/auto-build-check.sh

#!/usr/bin/env bash
set -euo pipefail

FILE_PATH="${1:-}"

if [ -z "$FILE_PATH" ]; then
    exit 0
fi

build_result=0

case "$FILE_PATH" in
    # TypeScript / JavaScript
    *.ts|*.tsx|*.js|*.jsx)
        build_typescript || build_result=$?
        ;;
    # Python
    *.py)
        build_python || build_result=$?
        ;;
    # Rust
    *.rs)
        build_rust || build_result=$?
        ;;
    # Go
    *.go)
        build_go || build_result=$?
        ;;
    # Swift
    *.swift)
        build_swift || build_result=$?
        ;;
    # C / C++
    *.c|*.cpp|*.cc|*.h|*.hpp)
```

```
        if [ -f "Makefile" ]; then
            make 2>&1 | tail -20 || build_result=$?
        elif [ -f "CMakeLists.txt" ]; then
            cmake --build build 2>&1 | tail -20 || build_result=$?
        fi
        ;;
    # Non-source files: no build needed
    *)
        exit 0
        ;;
esac

if [ $build_result -ne 0 ]; then
    echo ""
    echo "BUILD FAILED: Fix the errors above before continuing."
    exit 1
fi
```

Each language has its own build function that uses the appropriate toolchain:

```
# From .claude/hooks/auto-build-check.sh

# TypeScript / JavaScript project
build_typescript() {
    if command -v npx &>/dev/null && [ -f "tsconfig.json" ]; then
        npx tsc --noEmit 2>&1 | tail -20
    elif command -v npm &>/dev/null && [ -f "package.json" ]; then
        npm run build 2>&1 | tail -20
    else
        return 0
    fi
}

# Python project
build_python() {
    if command -v mypy &>/dev/null; then
        mypy "$FILE_PATH" 2>&1 | tail -20
    elif command -v python3 &>/dev/null; then
        python3 -m py_compile "$FILE_PATH" 2>&1
    fi
```

```
}

# Swift / Xcode project
build_swift() {
    local scheme="${XCODE_SCHEME:-}"
    local destination="${XCODE_DESTINATION:-}"

    if [ -n "$scheme" ] && [ -n "$destination" ]; then
        xcodebuild -scheme "$scheme" -destination "$destination" -quiet 2>&1 | tai
    elif command -v swift &>/dev/null && [ -f "Package.swift" ]; then
        swift build 2>&1 | tail -20
    else
        return 0
    fi
}
```

The key insight: when the hook exits with code 1, Claude Code surfaces it as an error that the agent must address before continuing. The agent cannot ignore a failed build. It cannot say "I will fix that later." It must fix it now, on this edit, before making the next one.

This single mechanism eliminated the cascading-build-failure problem that plagued my first thousand sessions. Before the hook, agents would make five edits, each building on assumptions from the previous one, and the fifth edit would reveal that the second edit was wrong. With the hook, the second edit fails immediately and gets fixed before the third edit begins.

The `tail -20` pattern. Every build function pipes through `tail -20`. This is deliberate – build tools often produce hundreds of lines of output, most of which is noise. The last 20 lines typically contain the actual error messages. Truncating the output keeps the agent focused on the relevant errors instead of being overwhelmed by verbose build logs.

The fallback `return 0` pattern. If the expected build tool is not installed, the function returns 0 (success) instead of failing. This prevents the hook from blocking work in environments where the toolchain is not fully configured. The alternative – failing when the tool is missing – would make the hook unusable in CI environments or containers where not every tool is installed.

Performance considerations. Each auto-build takes 15-45 seconds depending on the project. That sounds like a lot of overhead, but consider the alternative: a 20-minute debugging session to untangle cascading build failures. The hook turns a 20-minute problem into a 30-second feedback loop. The net time savings across a session is substantial.

A concrete before-and-after. In one session without the auto-build hook, an agent made 11 edits to implement a new feature. Edits 1-4 were correct. Edit 5 introduced a type error. Edits 6-11 built on the broken code from edit 5. When the agent finally tried to build, it faced 23 error messages across 7 files. It took 35 minutes to debug. With the auto-build hook, edit 5 would have failed immediately. The agent would have fixed the type error in 30 seconds. Edits 6-11 would have been built on correct code. Total time saved: approximately 34 minutes.

Layer 5: Pre-Commit Security Hook – The Last Line of Defense

The pre-commit hook scans every staged file for secrets, credentials, and sensitive data before any commit reaches the repository. It runs automatically when the agent executes `git commit`.

The Security Decision Tree

```
flowchart TD
    START["Staged File\nReady for Commit"] --> BINARY{"Is file\nbinary?"}
    BINARY -->|Yes| SKIP["Skip file\n(no scan)"]
    BINARY -->|No| EXISTS{"File\nexists?"}
    EXISTS -->|"No (deleted)"| SKIP
    EXISTS -->|Yes| FILETYPE{"Check\nfile extension"}

    FILETYPE -->|".sqlite .db"| BLOCK_DB["BLOCKED\nDatabase file"]
    FILETYPE -->|".env .env.*"| BLOCK_ENV["BLOCKED\nEnvironment file"]
    FILETYPE -->|".pem .key .p12 .pfx"| BLOCK_CERT["BLOCKED\nCertificate/key file"
    FILETYPE -->|"Source file"| CONTENT["Scan file\ncontent"]

    CONTENT --> API{"Match:\nsk-[a-zA-Z0-9]{'{'}20,{'}'}"}
    API -->|Yes| BLOCK_API["BLOCKED\nAPI key (OpenAI/Anthropic)"]
    API -->|No| AWS{"Match:\nAKIA[A-Z0-9]{'{'}16{'}'}"}
```

```
    AWS -->|Yes| BLOCK_AWS["BLOCKED\nAWS access key"]
    AWS -->|No| GH{"Match:\n(ghp_|ghu_|ghs_|gho_|ghr_)*"}

    GH -->|Yes| BLOCK_GH["BLOCKED\nGitHub token"]
    GH -->|No| GL{"Match:\nglpat-[a-zA-Z0-9-]{'{'}20,{'}'}"}

    GL -->|Yes| BLOCK_GL["BLOCKED\nGitLab token"]
    GL -->|No| BEARER{"Match:\nBearer [a-zA-Z0-9_-.]{'{'}20,{'}'}"}

    BEARER -->|Yes| BLOCK_BEARER["BLOCKED\nHardcoded Bearer token"]
    BEARER -->|No| PRIVKEY{"Match:\n-----BEGIN PRIVATE KEY-----"}

    PRIVKEY -->|Yes| BLOCK_KEY["BLOCKED\nPrivate key"]
    PRIVKEY -->|No| WARN_CHECKS["Warning Checks"]

    WARN_CHECKS --> PATH{"/Users/ or\n/home/ path?"}
    PATH -->|Yes| WARN_PATH["WARNING\nHardcoded absolute path"]
    PATH -->|No| PASS_CHECK{"password=\nor passwd=?"}

    PASS_CHECK -->|Yes| WARN_PASS["WARNING\nPossible hardcoded password"]
    PASS_CHECK -->|No| TODO{"TODO/FIXME/\nHACK markers?"}

    TODO -->|Yes| INFO_TODO["INFO\nN markers found"]
    TODO -->|No| CLEAN["CLEAN\nNo issues found"]

    BLOCK_DB & BLOCK_ENV & BLOCK_CERT & BLOCK_API & BLOCK_AWS & BLOCK_GH & BLOCK_G
    WARN_PATH & WARN_PASS & INFO_TODO --> ALLOW_WARN["COMMIT ALLOWED\nWith warning
    CLEAN --> ALLOW["COMMIT ALLOWED\nNo issues"]

    style REJECT fill:#ff4444,color:#fff
    style ALLOW_WARN fill:#ffaa00,color:#000
    style ALLOW fill:#44bb44,color:#fff
    style BLOCK_DB fill:#ff6666,color:#fff
    style BLOCK_ENV fill:#ff6666,color:#fff
    style BLOCK_CERT fill:#ff6666,color:#fff
    style BLOCK_API fill:#ff6666,color:#fff
    style BLOCK_AWS fill:#ff6666,color:#fff
    style BLOCK_GH fill:#ff6666,color:#fff
    style BLOCK_GL fill:#ff6666,color:#fff
    style BLOCK_BEARER fill:#ff6666,color:#fff
    style BLOCK_KEY fill:#ff6666,color:#fff
```

```
    style WARN_PATH fill:#ffcc00,color:#000
    style WARN_PASS fill:#ffcc00,color:#000
    style INFO_TODO fill:#66aaff,color:#000
    style CLEAN fill:#44bb44,color:#fff
```

Security decision tree

Security decision tree

The scan has three severity levels: BLOCKED (commit rejected), WARNING (commit proceeds but flagged), and INFO (informational markers logged).

Blocking Patterns

The blocking patterns – these reject the commit entirely:

```
# From .claude/hooks/pre-commit-check.sh
# claude-prompt-stack/.claude/hooks/pre-commit-check.sh

BLOCKED=0
WARNED=0

for file in $STAGED_FILES; do
    # Skip binary files and deleted files
    if file "$file" 2>/dev/null | grep -q "binary"; then continue; fi
    if [ ! -f "$file" ]; then continue; fi

    # OpenAI / Anthropic API keys
    if grep -nE 'sk-[a-zA-Z0-9]{20,}' "$file" 2>/dev/null; then
        echo "BLOCKED: Possible API key found in $file"
        BLOCKED=1
    fi

    # AWS Access Keys
    if grep -nE 'AKIA[A-Z0-9]{16}' "$file" 2>/dev/null; then
        echo "BLOCKED: AWS access key found in $file"
        BLOCKED=1
    fi
```

```
    # GitHub tokens
    if grep -nE '(ghp_|ghu_|ghs_|gho_|ghr_)[a-zA-Z0-9]{36,}' "$file" 2>/dev/null;
        echo "BLOCKED: GitHub token found in $file"
        BLOCKED=1
    fi

    # GitLab tokens
    if grep -nE 'glpat-[a-zA-Z0-9\-]{20,}' "$file" 2>/dev/null; then
        echo "BLOCKED: GitLab token found in $file"
        BLOCKED=1
    fi

    # Generic Bearer tokens (hardcoded)
    if grep -nE 'Bearer [a-zA-Z0-9_\-\.]{20,}' "$file" 2>/dev/null; then
        echo "BLOCKED: Hardcoded Bearer token found in $file"
        BLOCKED=1
    fi

    # Private keys
    if grep -nE '-----BEGIN (RSA |EC |OPENSSH )?PRIVATE KEY-----' "$file" 2>/dev/n
        echo "BLOCKED: Private key found in $file"
        BLOCKED=1
    fi
done
```

Each regex is carefully calibrated. The `sk-[a-zA-Z0-9]{20,}` pattern matches Anthropic and OpenAI API keys. The minimum length of 20 characters avoids false positives on short variable names that happen to start with `sk-`. The `AKIA` prefix is specific to AWS access keys – four uppercase letters followed by exactly 16 alphanumeric characters. The GitHub token prefixes (`ghp_`, `ghu_`, `ghs_`, `gho_`, `ghr_`) cover personal access tokens, user tokens, server tokens, OAuth tokens, and refresh tokens.

Sensitive File Types

It also catches sensitive file types that should never be committed:

```
# From .claude/hooks/pre-commit-check.sh
```

```
for file in $STAGED_FILES; do
    case "$file" in
        *.sqlite|*.sqlite3|*.db)
            echo "BLOCKED: Database file staged for commit: $file"
            BLOCKED=1
            ;;
        .env|.env.*|*.env)
            echo "BLOCKED: Environment file staged for commit: $file"
            BLOCKED=1
            ;;
        *.pem|*.key|*.p12|*.pfx)
            echo "BLOCKED: Certificate/key file staged for commit: $file"
            BLOCKED=1
            ;;
    esac
done
```

Warning Patterns

Warning patterns flag suspicious content but allow the commit to proceed:

```
# Hardcoded absolute paths
if grep -nE '/Users/[a-zA-Z]|/home/[a-zA-Z]' "$file" 2>/dev/null; then
    echo "WARNING: Hardcoded absolute path in $file"
    WARNED=1
fi

# Hardcoded passwords
if grep -niE '(password|passwd|pwd)\s*=\s*"[^"]+"|password\s*:\s*"[^"]+"' "$file";
    echo "WARNING: Possible hardcoded password in $file"
    WARNED=1
fi

# TODO/FIXME/HACK markers (informational)
if grep -ncE '(TODO|FIXME|HACK|XXX):' "$file" 2>/dev/null | grep -v '^0$' >/dev/nu
    count=$(grep -cE '(TODO|FIXME|HACK|XXX):' "$file" 2>/dev/null || echo "0")
    echo "INFO: $count TODO/FIXME markers in $file"
fi
```

The `/Users/` and `/home/` patterns catch the surprisingly common mistake of hardcoding absolute paths that work on your machine but break everywhere else. I have caught real API keys with this hook. During debugging sessions, it is natural to paste credentials into code temporarily. The hook ensures they never reach the repository, even when the agent (or you) forget to clean up.

The TODO/FIXME markers are informational only – they do not block the commit. But they serve a useful purpose: the agent sees the count and can decide whether to address them before committing. In practice, agents often fix trivial TODOs when they see the count is low (2-3 markers), which gradually reduces the project's technical debt.

How Layers 4 and 5 Interact

The auto-build hook (Layer 4) and the security hook (Layer 5) form a two-stage quality gate:

1. **Every edit** triggers a build check. If the build fails, the agent must fix it before continuing. This ensures every intermediate state is compilable.
2. **Every commit** triggers a security scan. If secrets are detected, the commit is rejected. This ensures no sensitive data reaches the repository.

The ordering matters. The build hook runs first (during editing), ensuring that by the time the agent tries to commit, the code already compiles. The security hook runs at commit time, when all changes are staged and the full delta is visible.

This two-stage approach means the agent cannot accumulate broken code across multiple edits (Layer 4 catches it edit-by-edit) and cannot slip secrets into a "fix build" commit (Layer 5 catches it at commit time).

Why two separate hooks instead of one combined hook? Because they have different activation patterns and different failure modes. The build hook needs to run after every edit – it catches errors incrementally. The security hook needs to see the complete set of staged changes – it catches patterns that only become dangerous in aggregate (e.g., a password that spans two lines in a config file). Combining them into one hook would either run security scans

too frequently (wasting time on every edit) or run build checks too infrequently (missing errors until commit time).

Layer 6: Skills – Composable Validation Workflows

Skills are markdown files that define reusable validation protocols. They are more than documentation – they are executable workflows that the agent follows step by step.

The `functional-validation` skill enforces evidence-based completion claims:

```
# From skills/functional-validation.md
# claude-prompt-stack/skills/functional-validation.md

## Workflow

### Step 1: Build the Real Application
Verify the build succeeds with zero errors. If it fails, fix the build first.

### Step 2: Start the Application
Wait for the application to be fully ready (health check, UI loaded).

### Step 3: Exercise the Feature
Interact with the feature through its actual interface:
- For web apps: Navigate to the page, click buttons, fill forms
- For APIs: Send real HTTP requests with curl
- For mobile apps: Navigate the real UI in the simulator

### Step 4: Capture Evidence
Collect at least one form of evidence:
- Screenshots, API responses, log output, terminal output

### Step 5: Verify Against Requirements
Compare captured evidence against the original requirements.

## Evidence Standards
| Claim                  | Minimum Evidence                          |
|------------------------|-------------------------------------------|
| "Feature works"        | Screenshot or API response showing it works |
```

```
| "Bug is fixed"         | Before/after evidence showing the fix       |
| "No regressions"       | Key existing features still function        |
| "Performance improved" | Measurable metrics (timing, memory, etc.)   |

## Anti-Patterns
- NEVER claim "it should work" without evidence
- NEVER create mock objects to simulate behavior
- NEVER write unit tests as a substitute for functional validation
- NEVER assume the feature works based on reading the code alone
```

The evidence standards table is the enforcement mechanism. Without it, agents claim completion with “the code looks correct.” With it, agents know they must produce a screenshot, API response, or measurable metric. The shift from “I think it works” to “here is proof it works” fundamentally changes the reliability of AI-generated code.

The anti-patterns section addresses a specific failure mode I call “proof by code reading.” An agent implements a feature, reads its own code, and concludes the feature works. It never builds the code. It never runs the application. It never interacts with the feature through the UI. The functional-validation skill makes this shortcut impossible by requiring step-by-step evidence collection.

Platform-Specific Skills

The iOS validation runner skill adds platform-specific steps for simulator-based verification:

```
# From skills/ios-validation-runner.md
# claude-prompt-stack/skills/ios-validation-runner.md

### Step 4: Navigate to Feature

Use one of these approaches (in order of reliability):

1. Deep links (if supported):
   xcrun simctl openurl [SIMULATOR_UDID] "[URL_SCHEME]://[ROUTE]"

2. Accessibility tree (most reliable for taps):
   # Get accessibility tree with exact coordinates
   idb_describe operation:all
```

```
    # Then tap using coordinates from the tree
    idb_tap [x] [y]

3. Direct state modification (for screenshots only):
   Temporarily modify @State defaults to auto-present the target view.

## Tips
- DerivedData path: Always use ~/Library/Developer/Xcode/DerivedData/,
  NOT a local build/ directory
- Deep link UUIDs: Must be LOWERCASE — uppercase causes failures
- Simulator gestures: Use idb_describe for accessibility coordinates,
  never guess pixels
```

Every tip in that list represents a debugging session that cost 30-60 minutes. "Deep link UUIDs must be LOWERCASE" – that one cost me two hours before I discovered that iOS URL handling is case-sensitive for UUID path components. Encoding it in a skill means no future session repeats that discovery.

Skill Composition

Skills compose hierarchically. The `functional-validation` skill provides the general framework; the iOS validation runner adds platform-specific mechanics. When both are invoked, the agent follows the general workflow structure and uses the platform-specific steps for the mechanics.

You can build skill hierarchies for any platform:

- `functional-validation` (general) + `ios-validation-runner` (iOS)
- `functional-validation` (general) + `web-validation-runner` (React/Vue)
- `functional-validation` (general) + `api-validation-runner` (REST/GraphQL)

The general skill never changes. The platform-specific skills encode the toolchain knowledge. When you switch from iOS to web development, you swap the platform skill but keep the same evidence standards.

Why Skills Are Not Just Documentation

A common reaction to skills is “that is just documentation.” It is not. Documentation tells humans what to do. Skills tell AI agents what to do, in a format they can follow step by step, with explicit verification points at each step.

The difference is in the granularity. Human documentation says “verify the feature works.” A skill says “run `curl -s http://localhost:9999/health`, verify the response contains `{"status":"ok"}`, then navigate to the feature page, capture a screenshot, and compare it against the expected behavior.” The agent needs the granular version because it cannot infer the intermediate steps.

Skills also have a critical property that documentation lacks: they are invokable. You can tell an agent “invoke the functional-validation skill” and it will follow the entire workflow, step by step, without skipping anything. You cannot do this with a README.

Writing Effective Skills

The best skills share four properties:

Sequential steps with verification gates. Each step produces a verifiable output. “Build the application” is verified by “exit code 0.” “Start the server” is verified by “health check returns 200.” “Navigate to the feature” is verified by “screenshot shows the expected UI.” If any gate fails, the skill workflow stops and the agent must fix the problem before continuing.

Explicit tool commands. Do not say “check the API.” Say “run `curl -s http://localhost:9999/api/v1/endpoint | python3 -m json.tool`.” The agent should not have to figure out which tool to use or how to invoke it. Provide the exact command.

Anti-pattern lists. Every skill should end with a list of things the agent must NOT do. “NEVER claim completion without a screenshot.” “NEVER create mock data to simulate the API response.” “NEVER skip the health check step.” Anti-patterns are the guardrails that prevent agents from finding creative shortcuts around the workflow.

Platform-specific escape hatches. When the standard approach fails (the deep link does not work, the health check returns 503, the simulator is not booted), the skill should provide fallback approaches. The iOS validation runner lists three approaches in priority order: deep links, accessibility tree taps, and direct state modification. The agent tries them in order until one works.

Layer 7: Project Memory – Cross-Session Knowledge

MEMORY.md persists knowledge across AI sessions. Every session starts by reading it. Every session can append to it. Over time, it becomes the institutional memory of the project.

The template provides structure:

```
# From MEMORY.md
# claude-prompt-stack/MEMORY.md

## Key Facts
- Project: [PROJECT_NAME]
- Build command: [BUILD_COMMAND]
- Port: [PORT_NUMBER]

## Architecture Decisions
<!-- Record important choices so future sessions don't re-debate them -->

## Known Pitfalls
<!-- Things that have gone wrong before -->

## Lessons Learned
<!-- Insights from debugging sessions, failed approaches -->

## Session Log
| Date | Summary | Outcome |
|------|---------|---------|
| [DATE] | [What was done] | [Result] |
```

In my production project, MEMORY.md grew to over 200 lines across hundreds of sessions. Here are real entries:

- "The old backend binary at `/Users/nick/ils/ILSBackend/` returns raw data – ALWAYS use the new binary at `/Users/nick/Desktop/ils-ios/`."
- "Deep link UUIDs must be LOWERCASE – uppercase causes failures."

- "Always call `process.waitUntilExit()` before accessing `process.terminationStatus` – otherwise throws NSInvalidArgumentException."
- "ClaudeCodeSDK uses RunLoop which NIO does not pump. Use direct Process + DispatchQueue instead."

Each of those entries represents a debugging session that cost 30-60 minutes. With memory, the agent learns from the first occurrence and never repeats the mistake. The compound value grows with every session – after 100 sessions, the memory file contains knowledge that would take a new engineer weeks to accumulate.

The difference between memory and rules. Rules (Layer 3) encode stable, design-time knowledge – things you know before the session starts. Memory (Layer 7) encodes runtime discoveries – things you learned during sessions. Rules rarely change. Memory grows continuously.

When memory contradicts rules. This should not happen, but when it does, rules win. Rules are curated and reviewed. Memory is accumulated and can contain stale entries. If a memory entry says "port 8080" but the rules file says "port 9999," the agent should follow the rules file. Memory is advisory; rules are authoritative.

Memory growth management. MEMORY.md can grow too large. My production project's memory hit 218 lines before I started splitting detailed content into separate topic files:

```
memory/
├── MEMORY.md              # Index and key facts (< 200 lines)
├── skill-inventory.md     # Detailed skill documentation
├── v35-validation-skills.md # Validation skill specifics
├── debugging.md           # Detailed debugging notes
└── architecture.md        # Architecture decision records
```

The main MEMORY.md stays concise and links to the detailed files. This keeps the primary context footprint small while preserving all the institutional knowledge. The 200-line limit is practical, not arbitrary – beyond 200 lines, agents start to lose attention on entries near the bottom of the file.

Memory maintenance. Every 50-100 sessions, review MEMORY.md and prune entries that are no longer relevant. Bugs that have been fixed, workarounds for issues that have been resolved, and decisions that have been superseded should be archived (moved to a separate file) or deleted. Stale memory wastes context and can mislead agents who follow outdated advice.

Memory Anti-Patterns

These are patterns I have seen in memory files that actively degrade agent performance:

The opinion memory. "React is better than Vue for this project." This is a preference, not a fact. Memory entries should be falsifiable observations, not subjective judgments. Replace with: "This project uses React 18 with concurrent features. All new components use functional components with hooks."

The stale workaround. "Use `--legacy-peer-deps` to work around the dependency conflict." This was valid six months ago. The dependency conflict was resolved in version 2.3.0. The agent still passes `--legacy-peer-deps` on every install because the memory says to. Stale workarounds are worse than no workaround because they mask whether the underlying issue has been fixed.

The overly specific memory. "On 2025-12-14 at 3:47pm, the build failed because the TypeScript compiler was version 5.2.2 and the project required 5.3.0." This level of detail is useless for future sessions. The useful entry is: "TypeScript version must match `engines.typescript` in package.json. Mismatched versions cause type errors in generic constraints."

The contradictory memory. Two entries in the same file: "Always use port 8080 for the backend" and "Backend port changed to 9999 to avoid conflicts with ralph-mobile." The agent encounters both and must guess which is current. The fix: when updating a fact, delete the old entry. Memory should have one source of truth per topic.

The unsigned memory. Entries without dates make it impossible to know if they are current. Every memory entry should include a date stamp. A memory entry from 6 months ago might be stale. A memory entry from yesterday is almost certainly current. The date is the agent's signal for how much to trust the entry.

The Memory Lifecycle

Memory entries follow a predictable lifecycle:

1. **Discovery.** A debugging session reveals something unexpected. "The backend binary at `/old/path/` returns raw data instead of API-wrapped responses." This costs 30-60 minutes to discover.
2. **Recording.** The entry is added to MEMORY.md under "Known Pitfalls" or "Lessons Learned." This takes 30 seconds.
3. **Reuse.** Future sessions read the entry and avoid the pitfall. Each reuse saves 30-60 minutes. Over 50 sessions, a single well-placed entry can save 10-20 hours.
4. **Promotion.** Entries that prove universally important get promoted to Layer 3 rules files. "Always use the new backend binary" becomes a rule in `project.md` with the correct path documented. The memory entry is simplified to a reference.
5. **Archival.** When the underlying issue is fixed (the old backend binary is deleted, the bug is patched, the migration is complete), the entry is no longer relevant. It gets moved to an archive file or deleted.

Not all entries reach step 4 or 5. Some remain in memory indefinitely because they are too specific for a rules file but too important to delete. "ClaudeCodeSDK uses RunLoop which NIO does not pump" – this is relevant only when someone tries to use the SDK in a Vapor project, which happens rarely but catastrophically when it does.

Memory as a Quality Signal

The rate at which memory grows is itself a useful metric. Fast memory growth (10+ new entries per session) suggests the project has undocumented complexity – the rules files (Layer 3) are missing critical domain knowledge. Slow memory growth (0-1 entries per session) suggests the stack is mature and the agent encounters few surprises.

If memory growth stalls completely (zero new entries for 20+ sessions), it could mean the project is stable – or it could mean the agent has stopped documenting discoveries. Add a periodic reminder to review and update memory as part of the validation workflow.

Layer Interactions: How Data Flows Through the Stack

The 7 layers are not independent – they form two interconnected data flows.

```
flowchart TD
    subgraph Upward["Data Flow Up (Runtime Feedback)"]
        direction BT
        U7["Layer 7: Memory\n(cross-session knowledge)"]
        U6["Layer 6: Skills\n(validation workflows)"]
        U5["Layer 5: Security Hook\n(blocked/warned patterns)"]
        U4["Layer 4: Auto-Build Hook\n(build pass/fail)"]
        U3["Layer 3: Rules Files\n(domain constraints)"]
        U2["Layer 2: Project CLAUDE.md\n(project mandates)"]
        U1["Layer 1: Global CLAUDE.md\n(universal constitution)"]

        U7 -->|"Pitfalls & lessons\nfeed into rules"| U3
        U6 -->|"Validation results\nconfirm mandate compliance"| U2
        U5 -->|"Blocked patterns\nreinforce security rules"| U3
        U4 -->|"Build failures\nforce immediate fix"| U2
    end

    subgraph Downward["Data Flow Down (Constraint Propagation)"]
        direction TB
        D1["Layer 1: Global CLAUDE.md\n'Never write mocks'"]
        D2["Layer 2: Project CLAUDE.md\n'One change, one verify'"]
        D3["Layer 3: Rules Files\n'Port 9999, API prefix /api/v1'"]
        D4["Layer 4: Auto-Build Hook\n'Build after every edit'"]
        D5["Layer 5: Security Hook\n'Block sk-*, AKIA*, ghp_*'"]
        D6["Layer 6: Skills\n'Follow validation workflow'"]
        D7["Layer 7: Memory\n'Remember: UUIDs must be lowercase'"]

        D1 -->|"Constrains all\nproject behavior"| D2
        D2 -->|"Narrows scope\nto project context"| D3
        D3 -->|"Provides constants\nfor build commands"| D4
        D4 -->|"Ensures code compiles\nbefore commit"| D5
        D5 -->|"Ensures no secrets\nbefore validation"| D6
        D6 -->|"Records outcomes\nfor future sessions"| D7
    end

    subgraph Lifecycle["Session Lifecycle"]
        direction LR
```

```
        SL["Session Load\n(Layers 1–3 + 7)"]
        ED["Each Edit\n(Layer 4 fires)"]
        CM["Each Commit\n(Layer 5 fires)"]
        VL["Validation\n(Layer 6 invoked)"]
        EN["Session End\n(Layer 7 updated)"]

        SL --> ED --> CM --> VL --> EN
    end
```

Layer interactions

Layer interactions

Downward flow (constraint propagation): Layer 1 sets universal mandates. Layer 2 narrows to project specifics. Layer 3 provides domain knowledge. Layer 4 enforces build correctness. Layer 5 enforces security. Layer 6 provides validation workflows. Layer 7 records outcomes.

Upward flow (runtime feedback): When a build fails (Layer 4), the error feeds back into the agent's context, which is shaped by Layer 2's "one change, one verify" mandate. When a security scan blocks a commit (Layer 5), the blocked pattern reinforces the security rules in Layer 3. When a debugging session discovers a new pitfall, it gets recorded in Layer 7's memory, which feeds future sessions' understanding of Layer 3's domain context.

The session lifecycle: Layers 1-3 and 7 load at session start. Layer 4 fires on every edit. Layer 5 fires on every commit. Layer 6 is invoked during validation. Layer 7 updates at session end.

Cross-session evolution. Over time, the stack improves itself. Memory entries that prove universally useful get promoted to rules files. Rules that prove too broad get narrowed based on memory entries that document edge cases. The auto-build hook's language detection gets updated when a new file type is introduced. The security patterns get tightened when false positives are reported in memory. Each session's feedback makes the next session more reliable.

Concrete Layer Interaction Examples

To make the data flow tangible, here are three scenarios where multiple layers interact within a single edit:

Scenario 1: Agent edits a Swift file in an iOS project. 1. Layer 3 (rules file `project.md`) told the agent that the build command is `xcodebuild -project ILSApp.xcodeproj -scheme ILSApp -destination 'id=50523130'`. 2. The agent makes its edit. 3. Layer 4 (auto-build hook) fires. The hook reads the file path, detects `*.swift` under `ILSApp/`, and runs the xcodebuild command from Layer 3. 4. The build fails: "Cannot find type 'ThemeSnapshot' in scope." 5. The agent reads the error, checks Layer 7 (memory) which says "ThemeSnapshot is a concrete struct, not a protocol – import it from Theme/ThemeSnapshot.swift." 6. The agent adds the missing import, Layer 4 fires again, and the build passes.

Without Layer 4, the agent would have continued editing other files, compounding the error. Without Layer 7, the agent would have spent 5 minutes searching for the type instead of knowing the fix immediately. Without Layer 3, the build hook would not have known which Xcode scheme to use.

Scenario 2: Agent prepares to commit code containing a hardcoded path. 1. The agent finishes a feature and runs `git commit`. 2. Layer 5 (security hook) scans staged files and finds `/Users/nick/Desktop/` in a constant. 3. The hook emits a WARNING (hardcoded path, not a blocked pattern). 4. Layer 3 (rules file `project.md`) says "Common Pitfalls: hardcoded `/Users/` paths cause failures on other machines." 5. The agent replaces the hardcoded path with a runtime-resolved path and re-commits. 6. Layer 5 runs again, finds no issues, and the commit succeeds.

Scenario 3: Agent is asked to add a premium feature. 1. Layer 2 (project CLAUDE.md) loads at session start. 2. Layer 3 (rules file `feature-gate.md`) says "NEVER add `if isPremium` checks directly in views – use `FeatureGateView`." 3. The agent writes the feature using `FeatureGateView` as instructed. 4. Layer 4 (auto-build hook) verifies the build compiles. 5. Layer 6 (skill `functional-validation`) runs and verifies the feature works for both premium and free users by running the real app. 6. Layer 7 (memory) records "FeatureGateView wrapping pattern verified for chatExport feature."

Three layers provided knowledge (2, 3, 7), one enforced correctness (4), one validated the result (6). No single layer could have produced this outcome alone. The stack's value is cumulative.

The Override Hierarchy: What Wins When Layers Contradict

When two layers give conflicting instructions, which one wins? This is the precedence order, from highest to lowest:

```
flowchart TD
    subgraph Precedence["Override Hierarchy (Highest to Lowest)"]
        direction TB
        H["Hook Exit Code\n(Layer 4-5)\nBINARY: pass or fail\nCannot be overridden
        S["Security Scan\n(Layer 5)\nBLOCK = commit rejected\nNo exceptions"]
        SK["Skill Workflow\n(Layer 6)\nStep-by-step protocol\nAgent must follow se
        PR["Project CLAUDE.md\n(Layer 2)\nProject-specific mandates\nOverrides glo
        RU["Rules Files\n(Layer 3)\nDomain knowledge\nSupplements CLAUDE.md"]
        GL["Global CLAUDE.md\n(Layer 1)\nUniversal constitution\nBase behavioral r
        ME["Memory\n(Layer 7)\nCross-session knowledge\nAdvisory, not mandatory"]
    end

    H -->|"Overrides everything:\nfailed build blocks\nall further action"| S
    S -->|"Overrides agent intent:\nblocked pattern =\nno commit regardless"| SK
    SK -->|"Overrides ad-hoc approach:\nagent follows prescribed\nworkflow steps"|
    PR -->|"Overrides global defaults:\nproject says port 9999\nnot global default
    RU -->|"Supplements project rules:\nadds domain detail\nwithout contradicting"
    GL -->|"Provides baseline:\nagent behavior when\nno override exists"| ME

    subgraph Conflict["When Layers Conflict"]
        direction LR
        C1["Global says: 'Use Opus for reviews'\nProject says: 'Use Sonnet for all
        C1R["Resolution: Project wins\n(Layer 2 > Layer 1)"]
        C1 --> C1R

        C2["Rules say: 'Build with npm'\nHook detects: TypeScript file"]
        C2R["Resolution: Hook runs npm build\n(Layer 4 uses Layer 3 context)"]
        C2 --> C2R

        C3["Memory says: 'Skip linting'\nSecurity hook: scans anyway"]
        C3R["Resolution: Hook always runs\n(Layer 5 > Layer 7)"]
        C3 --> C3R
    end

    style H fill:#ff4444,color:#fff
```

```
    style S fill:#ff6644,color:#fff
    style SK fill:#ff8844,color:#fff
    style PR fill:#ffaa44,color:#000
    style RU fill:#ffcc44,color:#000
    style GL fill:#44aa44,color:#fff
    style ME fill:#4488cc,color:#fff
```

Override hierarchy

Override hierarchy

The key principle: **binary enforcement overrides advisory guidance.** Hooks produce binary pass/fail results. CLAUDE.md and rules files produce advisory text. A hook failure is not something the agent can reason its way around – the exit code blocks further action at the tool level.

Concrete conflict examples:

1. **Global says "Use Opus for reviews" / Project says "Use Sonnet for all tasks."** Resolution: Project wins. Layer 2 overrides Layer 1 for project-specific settings. The global file sets defaults; the project file customizes them.
2. **Memory says "Skip the linting step" / Security hook blocks the commit.** Resolution: Hook always runs. Memory is advisory (Layer 7, lowest precedence). The security hook is binary enforcement (Layer 5, second-highest). The agent cannot skip a hook just because memory says to.
3. **Rules say "Build with npm" / Hook detects a TypeScript file.** Resolution: Cooperative, not conflicting. The rules file provides the knowledge that npm is the correct build tool. The hook uses that knowledge to run the right build command. Layer 3 informs Layer 4.
4. **Skill says "capture a screenshot" / Build hook says the build is broken.** Resolution: Hook wins. The build must pass before validation can proceed. The skill's workflow is paused until the build is fixed.

Debugging Prompt Stack Issues

When the stack is not working as expected, here is how to diagnose.

Symptom: Agent ignores a rule from CLAUDE.md. Likely cause: The agent's context window is full. Rules at the top of CLAUDE.md get more attention than rules at the bottom. Move critical rules to the top of the file. If the file exceeds ~500 lines, split it and put the most important rules in the first 200 lines.

Symptom: Auto-build hook fires but does not catch errors. Likely cause: The build command in the hook does not match the project's actual build system. Check that the `case` statement routes the file extension to the correct build function, and that the build function uses the right toolchain. Test manually: `bash .claude/hooks/auto-build-check.sh path/to/edited-file.ts`

Symptom: Security scan produces false positives. Likely cause: The regex patterns are too broad. The `sk-` pattern, for example, can match legitimate variable names like `sk-navigation`. Tighten the minimum length (`{20,}` to `{30,}`) or add exception patterns. You can also exclude specific files from scanning by adding an allowlist at the top of the script.

Symptom: Agent does not follow skill workflow. Likely cause: The skill was not invoked. Skills must be explicitly invoked – they are not auto-loaded like rules files. Check that the agent's instructions include a skill invocation mandate. In my setup, Layer 2 (CLAUDE.md) contains: "BEFORE any task, check available skills. If a skill might apply, invoke it first."

Symptom: Memory file grows too large and agent ignores entries. Likely cause: Memory exceeds the agent's effective attention span (roughly 200 lines). Split the memory file into an index file (MEMORY.md with key facts) and topic-specific files (memory/*.md). Keep the main file under 200 lines.

Symptom: Hook runs but agent does not react to failures. Likely cause: The hook's exit code is incorrect. A hook must exit with code 1 to signal failure – exit 0 means success even if error messages are printed. Verify that the script's final `exit` statement uses the correct code. Test with: `bash .claude/hooks/auto-build-check.sh bad-file.ts; echo $?`

Symptom: Agent works fine on the first task but fails on the second. Likely cause: State pollution. The first task modified a file that the second task depends on, and the agent's context

still references the pre-modification version. This is rare with Claude Code (which re-reads files) but can happen with cached tool results. Restart the session between independent tasks, or explicitly tell the agent to re-read modified files.

Symptom: New team member's sessions fail but yours work. Likely cause: The global CLAUDE.md (Layer 1) is in your home directory and has project-specific content that should be in Layer 2 or 3. Everything in `~/.claude/CLAUDE.md` is personal. Everything in the project's `CLAUDE.md` and `.claude/rules/` is shared via version control. Audit Layer 1 for content that belongs in Layer 2 or 3.

The Debugging Checklist

When something goes wrong, check these in order:

1. **Is the hook executable?** Run `ls -la .claude/hooks/` and verify the scripts have execute permission.
2. **Does the hook run standalone?** Run `bash .claude/hooks/auto-build-check.sh path/to/file.swift` outside of Claude Code.
3. **Does the hook produce the right exit code?** Run the hook and check `echo $?` – it should be 0 for success and 1 for failure.
4. **Is `settings.local.json` valid JSON?** A trailing comma or missing bracket will silently disable all hooks.
5. **Is the matcher pattern correct?** `Edit|Write|MultiEdit` catches file edits. `Bash` catches shell commands. A typo in the matcher means the hook never fires.
6. **Is the CLAUDE.md loading?** Add a unique phrase to CLAUDE.md and ask the agent to repeat it. If the agent cannot, the file is not loading.

The Stack in CI/CD Pipelines

The prompt stack is not just for interactive development. It transfers directly to CI/CD pipelines where AI agents run without human oversight – exactly where reliability matters most.

Replicating the stack in CI. The key insight is that CI already has the infrastructure for Layers 4 and 5. Build verification (Layer 4) is your CI build step. Security scanning (Layer 5) is your

secret detection step. The missing pieces are Layers 1-3 (agent configuration) and Layers 6-7 (skills and memory).

In a GitHub Actions workflow, the stack maps like this:

```
# .github/workflows/ai-pipeline.yml
jobs:
  ai-execution:
    steps:
      # Layer 1-3: Configuration (already in repo)
      - uses: actions/checkout@v4

      # Layer 4: Build verification after each agent step
      - name: Agent executes task
        run: |
          claude --print -p "Read CLAUDE.md, then implement the task"
          # Hook equivalent: verify build after execution
          npm run build

      # Layer 5: Security scan before merge
      - name: Security scan
        run: |
          # Replicate pre-commit hook logic
          bash .claude/scripts/pre-commit-check.sh

      # Layer 6: Validation skill equivalent
      - name: Functional validation
        run: |
          npm run start &
          sleep 5
          curl -s http://localhost:3000/health | jq .status

      # Layer 7: Memory update
      - name: Record outcome
        if: always()
        run: |
          echo "$(date): Task result=${{ job.status }}" >> .omc/ci-memory.log
```

The critical difference in CI: no interactive recovery. In an interactive session, when Layer 4 catches a build failure, the agent can read the error and fix it immediately. In CI, you need to structure the pipeline so that failures are captured, categorized, and either auto-retried or escalated to a human. The fix cycle from the worktree pipeline (Post 6) applies here – the CI pipeline can re-run the agent with the error output appended to its context.

Layer 7 in CI is where organizations underinvest. Every failed CI run produces diagnostic information – which files were touched, what errors occurred, which patterns caused failures. Recording this into a structured memory that subsequent runs can reference prevents the same failure from happening twice. Without CI memory, your AI agents rediscover the same pitfalls on every run.

Layer 3 rules files are the highest-leverage CI addition. Adding a `.claude/rules/ci.md` file that documents CI-specific constraints (available environment variables, service containers, artifact paths, deployment targets) prevents agents from making assumptions that work locally but fail in CI. The difference between a local build and a CI build is entirely captured in Layer 3 context.

Common Mistakes When Building Your Stack

Over hundreds of sessions helping teams adopt the stack, I have seen the same mistakes repeated. Here are the patterns and their fixes.

Mistake 1: Writing a 1,000-line CLAUDE.md. The agent cannot pay attention to 1,000 lines of instructions. The most effective CLAUDE.md files are 100-300 lines. Move detailed domain knowledge into `.claude/rules/` files. Move detailed pitfalls into MEMORY.md. Keep CLAUDE.md focused on the highest-impact rules. I tested this empirically: a 150-line CLAUDE.md with 6 rules files produced better agent compliance than a 900-line monolithic CLAUDE.md, because the agent loaded domain knowledge only when relevant files were edited rather than processing everything upfront.

Mistake 2: Making the auto-build hook too slow. If the hook takes 60+ seconds, the agent's workflow becomes painfully slow. Optimize build commands: use `--noEmit` for TypeScript (no output generation), `py_compile` for Python (syntax check only, faster than mypy), and `-`

`quiet` for Xcode (suppresses verbose output). The hook should complete in under 30 seconds for most edits. Beyond 45 seconds, agents begin to exhibit a specific bad behavior: they batch multiple changes into a single edit to avoid triggering the hook multiple times, which defeats the purpose of incremental verification.

Mistake 3: Pre-populating MEMORY.md with assumptions. Let memory grow organically from real sessions. Pre-populated memory entries are often wrong or irrelevant. Real memory entries come from real problems – they are battle-tested knowledge. Assumed entries are guesses. One team pre-populated 40 memory entries based on their internal documentation. After 30 sessions, 28 of those entries had never been referenced by the agent. The 12 entries that were useful had been rewritten based on actual session outcomes. The pre-population was wasted effort.

Mistake 4: Forgetting to document the hook system itself. The `auto-build-hook.md` rules file tells the agent that hooks exist and what they do. Without this file, the agent does not understand why builds are running automatically and may try to "fix" the behavior by disabling the hook. The meta-documentation – documentation about the documentation system – is essential. I have seen agents attempt to delete the hook script because they interpreted the automatic build as "unexpected behavior." The rules file prevents this by making the hook a known part of the workflow.

Mistake 5: Using the same model for everything. The auto-build hook catches errors. It does not need sophisticated reasoning. The security scan matches regex patterns. It does not need a language model at all. The QA review needs deep reasoning about code correctness. Match the tool to the task. Not every layer needs an AI model, and not every AI model needs to be the most powerful one.

Mistake 6: Putting environment-specific values in Layer 2. Database connection strings, API endpoints, file paths that differ between machines – these do not belong in the project CLAUDE.md. They change per developer and per environment. Use Layer 3 rules files with naming like `local-overrides.md` (gitignored) for machine-specific values, and keep Layer 2 strictly for universal project truths.

Mistake 7: Not versioning the stack. The `.claude/` directory should be committed to version control (except machine-specific overrides). When the stack is not versioned, team members end up with different hook configurations, different rules files, and different security

patterns. The result is inconsistent agent behavior across the team – the same prompt produces different results depending on who runs it. Version the stack. Review changes to it in pull requests just like any other code.

Mistake 8: Treating Layer 7 as a log file. Memory is not a chronological log of everything the agent did. It is curated knowledge that improves future sessions. If your MEMORY.md grows beyond 200 lines, prune it. Move detailed entries into topic-specific files. Keep the main memory file as an index of the most important lessons, not a transcript of past sessions.

Building Your Own Prompt Stack: Step by Step

The companion repo includes a setup script that creates the entire stack in one command:

```
# From setup.sh
# claude-prompt-stack/setup.sh

bash setup.sh --target /path/to/your/project

# Creates:
# Layer 2: CLAUDE.md
# Layer 3: .claude/rules/ (9 files)
# Layer 4: .claude/hooks/auto-build-check.sh
# Layer 5: .claude/hooks/pre-commit-check.sh
# Layer 6: skills/ (2 files)
# Layer 7: MEMORY.md
```

The setup script is non-destructive – it skips files that already exist:

```
# From setup.sh

create_file() {
    local dest="$1"
    local src="$2"
    local full_dest="$TARGET_DIR/$dest"

    mkdir -p "$(dirname "$full_dest")"
```

```
    if [ -f "$full_dest" ]; then
        echo "  SKIP  $dest (already exists)"
        SKIPPED=$((SKIPPED + 1))
    else
        cp "$src" "$full_dest"
        echo "  CREATE $dest"
        CREATED=$((CREATED + 1))
    fi
}
```

You can preview what would be created with `--dry-run`:

```
bash setup.sh --target ./my-project --dry-run
```

After running the setup, customize in this order:

Step 1: Edit CLAUDE.md. Replace every `[PLACEHOLDER]` with your project's actual values. The build commands, port numbers, API prefixes, and common pitfalls are the highest-impact customizations.

Step 2: Edit `.claude/rules/project.md`. Fill in your tech stack, directory structure, and architecture notes. This is the file the agent references most frequently when making decisions.

Step 3: Configure the auto-build hook. In `.claude/hooks/auto-build-check.sh`, ensure the `case` statement routes your project's file extensions to the correct build functions. For Swift/Xcode projects, set the `XCODE_SCHEME` and `XCODE_DESTINATION` environment variables. For TypeScript projects, verify that `tsconfig.json` exists in the project root.

Step 4: Review security patterns. In `.claude/hooks/pre-commit-check.sh`, add any project-specific patterns (e.g., your company's internal token format) and remove any patterns that produce false positives in your codebase.

Step 5: Start using it. Launch a Claude Code session. The stack is operational. Layers 1-3 and 7 load automatically. Layers 4-5 fire on relevant events. Layer 6 is invoked when you or the agent triggers a skill.

Step 6: Let memory grow organically. Do not try to pre-populate MEMORY.md. Let it grow through real sessions. After each debugging session that reveals a non-obvious insight, add it to the Known Pitfalls or Lessons Learned section. After 20-30 sessions, the memory file will contain the project's most valuable institutional knowledge.

Verification After Setup

After completing the setup, verify each layer is working before starting real work:

```
# Layer 2-3: Configuration loads correctly
claude --print "What project am I working on? \
  What is the build command? What port does the backend use?" \
  2>&1 | head -20

# Layer 4: Auto-build hook fires on edit
# (Make a trivial edit to any source file and watch for build output)

# Layer 5: Security hook fires on commit
echo 'sk-test-fake-key' >> /tmp/test-secret.txt
git add /tmp/test-secret.txt
git commit -m "test" 2>&1 | grep -i "blocked\|warning"
git reset HEAD /tmp/test-secret.txt

# Layer 7: Memory loads
claude --print "What do you know from project memory \
  about known pitfalls?" 2>&1 | head -20
```

If Layer 4 does not fire on edits, check that `settings.local.json` has the correct `PostToolUse` hook configuration and that the script path is absolute. If Layer 5 does not catch the test secret, check that the `PreToolUse` hook matches `git commit` commands. These two hooks are the most common failure points during initial setup.

The most common setup failure: a JSON syntax error in `settings.local.json`. A single missing comma silently disables all hooks with no error message. Validate the JSON before trusting the hooks:

```
python3 -c "import json; json.load(open('.claude/settings.local.json'))" \
  && echo "JSON valid" || echo "JSON INVALID"
```

Measuring Stack Effectiveness

How do you know the stack is working? Track these metrics across sessions:

Build failure recovery time. With the auto-build hook, this should be under 60 seconds. Without it, cascading build failures average 15-25 minutes to resolve. If recovery time exceeds 2 minutes, the hook's build command may be misconfigured.

Secret leak rate. Should be zero. If a secret reaches your git history even once, the security hook needs its patterns updated. Track by periodically scanning `git log` for known patterns.

Scope creep incidents. Count how often an agent touches files outside its assigned scope. With the "No Scope Creep" rule in Layer 2, this should be rare. If it exceeds 10% of sessions, the rule needs to be more prominent or more specific.

Validation evidence quality. With the functional-validation skill, agents should produce screenshots, API responses, or log output as proof of completion. Track how often agents claim "it works" without evidence. If this exceeds 5% of completion claims, the skill invocation mandate needs strengthening.

Memory reuse rate. Track how often a session benefits from a memory entry that was written in a previous session. This is harder to measure but can be approximated by counting memory references in session logs. A healthy project memory prevents at least 2-3 repeated mistakes per session.

The Stack Across Different Project Types

The 7-layer stack was developed on an iOS/Swift project, but the architecture applies to any language and framework. Here is how the layers adapt to different project types.

Python/FastAPI backend project.

Layer	iOS/Swift Implementation	Python/FastAPI Adaptation
Layer 1	Global model routing	Same — model routing is language-agnostic
Layer 2	`swift build`, port 9999	`uvicorn main:app`, port 8000
Layer 3	Xcode schemes, DerivedData paths	virtualenv activation, alembic migrations
Layer 4	`xcodebuild -quiet`	`mypy file.py` or `python -m py_compile`
Layer 5	Same security patterns	Add `DJANGO_SECRET_KEY` pattern
Layer 6	`ios-validation-runner`	`api-validation-runner` (curl-based)
Layer 7	iOS simulator pitfalls	Database migration pitfalls

TypeScript/React frontend project.

Layer	iOS/Swift Implementation	TypeScript/React Adaptation
Layer 1	Same	Same
Layer 2	Simulator UDID, deep links	`npm run dev`, `localhost:3000`
Layer 3	SwiftUI navigation patterns	React Router patterns, state management

Layer	iOS/Swift Implementation	TypeScript/React Adaptation
Layer 4	`xcodebuild`	`npx tsc --noEmit`
Layer 5	Same	Add `NEXT_PUBLIC_` env var patterns
Layer 6	Simulator screenshots	Playwright browser screenshots
Layer 7	iOS-specific pitfalls	CSS/hydration/SSR pitfalls

Rust/CLI tool project.

Layer	iOS/Swift Implementation	Rust/CLI Adaptation
Layer 1	Same	Same
Layer 2	Xcode project structure	Cargo workspace structure
Layer 3	iOS framework patterns	Ownership/lifetime patterns
Layer 4	`xcodebuild`	`cargo check` (faster than `cargo build`)
Layer 5	Same	Same
Layer 6	Simulator validation	CLI output validation (expected stdout)
Layer 7	iOS-specific pitfalls	Borrow checker pitfalls

The pattern is consistent: Layers 1, 5, and 7 are nearly identical across project types. Layer 4 changes the build command. Layers 2-3 and 6 encode project-specific knowledge. The architecture is portable even though the content is not.

A Real Session: The Stack in Action

Here is what a typical 30-minute session looks like with all 7 layers active. The task: add a new API endpoint to the iOS backend that returns project statistics.

Minute 0: Session starts. Claude Code loads Layers 1-3 and 7 automatically. The agent knows: this is a Swift/Vapor project, the backend runs on port 9999, the API prefix is `/api/v1`, navigation uses `activeScreen` not `selectedTab`, and the old backend binary at a different path should never be used (from memory).

Minute 1: Agent reads the task. "Add a GET /api/v1/stats/projects endpoint that returns total projects, projects by language, and average project size." The agent checks for applicable skills (per Layer 1's skill invocation mandate) and identifies `functional-validation` as relevant.

Minute 3: Agent creates the route. It adds a new `StatsController.swift` file in `Sources/ILSBackend/Controllers/`. It knows the correct directory because Layer 3's `project.md` documents the directory structure.

Minute 3.5: Layer 4 fires. The auto-build hook detects a `.swift` file was edited. It runs `swift build`. Build succeeds. The agent continues.

Minute 5: Agent adds the route registration. It modifies `routes.swift` to register the new controller. Layer 4 fires again. Build succeeds.

Minute 8: Agent implements the query logic. It writes the Fluent ORM query to count projects by language. Layer 4 fires. Build fails – the agent used `Project.query(on: db).group(.count)` but the correct Fluent syntax is `Project.query(on: db).count()`. The error message from the hook is clear: "value of type 'QueryBuilder' has no member 'group'."

Minute 9: Agent fixes the query. Layer 4 fires. Build succeeds.

Minute 12: Agent invokes the functional-validation skill. Following the skill's workflow: Step 1 (build) already confirmed. Step 2: the agent checks if the backend is running with `lsof -i :9999 -P -n`. It is. Step 3: the agent sends `curl -s http://localhost:9999/api/v1/stats/projects | python3 -m json.tool` and gets the response. Step 4: the agent captures the curl output as evidence.

Minute 14: Agent prepares to commit. It stages the new file and the modified routes file. Layer 5 fires – the pre-commit security hook scans both files. No secrets found. No hardcoded paths. One INFO marker (a TODO comment in the new controller). Commit proceeds.

Minute 15: Session complete. The agent reports completion with evidence: the curl output showing the new endpoint returning real data. Memory (Layer 7) does not need an update – no new pitfalls were discovered.

Total time: 15 minutes. Build errors caught: 1 (fixed in 60 seconds). Security issues: 0. Evidence captured: yes. Without the stack, the same task would likely have taken 25-35 minutes – the build error would have cascaded into subsequent edits, and the agent would have reported "done" without testing the endpoint.

What Would Have Happened Without the Stack

To appreciate the stack's value, consider the same task without each layer:

Without Layer 3 (rules files): The agent would not have known the correct directory structure. It would have spent 3-5 minutes reading source files to figure out where controllers live, what the Fluent ORM conventions are, and what the existing route registration pattern looks like. It might have placed the file in the wrong directory or used a different naming convention.

Without Layer 4 (auto-build hook): The agent would have written all the code, then attempted to build at the end (or not at all). The Fluent syntax error would have cascaded – the agent would have continued writing code that depended on the broken query. By the time it discovered the build error, 3-4 additional files would need fixing instead of 1.

Without Layer 5 (security hook): Not relevant for this specific task. But in a session where the agent uses a test API key during debugging, the security hook is the difference between that key ending up in git history forever and being caught at commit time.

Without Layer 6 (skills): The agent would have reported completion without testing the endpoint. "I created the controller, registered the route, and the code compiles. The feature should work." Should. The functional validation skill replaces "should" with "does" by requiring the agent to actually call the endpoint and capture the response.

Without Layer 7 (memory): The agent might have used the old backend binary path (a known pitfall documented in memory), wasting 10 minutes debugging "connection refused" errors

before realizing it was talking to the wrong server.

Each layer's contribution is modest in isolation – a few minutes saved here, an error caught there. Together, they cut the session time in half and eliminated the possibility of shipping untested code.

The Compound Effect

Here is why the stack works as a system and not just a collection of tips:

The AI cannot forget the build command (Layer 3 documents it, Layer 4 runs it automatically).

The AI cannot skip validation (Layer 2 mandates it, Layer 6 provides the workflow).

The AI cannot ship broken code (Layer 4 catches it on every single edit).

The AI cannot leak secrets (Layer 5 blocks them at commit time with pattern matching for `sk-*`, `AKIA*`, `ghp_*`, `glpat-*`, and Bearer tokens).

The AI cannot repeat past mistakes (Layer 7 remembers them across sessions).

The AI cannot waste time guessing (Layer 3 provides the answers to questions the agent would otherwise have to figure out by reading source files).

The AI cannot scope-creep (Layer 2 explicitly forbids it with behavioral rules).

The AI cannot ignore project conventions (Layer 3 makes them explicit, leaving no room for the agent to invent its own patterns).

The AI cannot misuse premium features (Layer 3's feature gate rules enforce the correct pattern, and Layer 4 catches compilation errors from incorrect usage).

No single layer achieves this. Layer 4 without Layer 3 knows that the build failed but not why. Layer 6 without Layer 2 provides a validation workflow but no mandate to follow it. Layer 7 without Layer 5 remembers past mistakes but cannot prevent the current one. The compound effect is not additive – it is multiplicative. Each layer's effectiveness is amplified by the others.

Quantifying the compound effect. I measured session outcomes across 500 sessions, 250 with the full stack and 250 without (early sessions before the stack was built). The results:

Metric	Without Stack	With Stack	Improvement
Build errors at commit time	34% of sessions	0% of sessions	100% elimination
Secret leaks caught pre-push	0 (none caught)	12 blocked	12 prevented incidents
Agent scope creep (>5 unrelated files)	28% of sessions	3% of sessions	89% reduction
“Works on first try” rate	41% of features	78% of features	90% improvement
Average session duration	42 minutes	23 minutes	45% faster
Sessions requiring manual cleanup	22%	4%	82% reduction

The 45% reduction in session duration is the metric that matters most for day-to-day productivity. Twenty minutes saved per session, across hundreds of sessions, is hundreds of hours. The stack pays for its setup cost in the first week of use.

No single layer provides all of this. Layer 4 alone catches build failures but does not prevent scope creep. Layer 7 alone remembers pitfalls but cannot block secrets. The defense-in-depth architecture means each layer catches the failures that slip through the layers above.

The compound effect is not additive – it is multiplicative. Each layer reduces a specific failure rate. When seven layers each reduce their respective failure rates by 80-95%, the combined system reduces the overall failure rate by orders of magnitude. The stack does not make AI agents perfect. It makes them reliable enough to trust with real work.

What I Would Tell Myself 8,000 Sessions Ago

Start with Layer 4 (auto-build hook). It has the highest impact-to-effort ratio. A 149-line bash script that runs after every edit eliminated more bugs than any other single intervention.

Then add Layer 5 (pre-commit security). Another 149-line script that runs before every commit. It costs nothing and prevents catastrophic mistakes.

Then build out Layers 1-3 (CLAUDE.md and rules). This is where you encode your project's institutional knowledge so the agent stops guessing.

Layers 6 and 7 (skills and memory) grow organically as you work. Do not try to write them all upfront. Let them emerge from real sessions and real failures.

The total effort to set up the full stack: about 2 hours. The total time saved across the next 100 sessions: conservatively 50-100 hours. That is the real return on investment of the prompt engineering stack.

The Stack's Evolution Over Time

The prompt stack is not static. It evolves through three phases as you use it.

Phase 1: Foundation (Sessions 1-20). You set up Layers 1-5 using the companion repo templates. The auto-build hook catches your first cascading build failure. The security hook blocks your first accidental API key commit. MEMORY.md has 10-20 entries, mostly key facts and the first few pitfalls discovered during setup. The stack already pays for itself.

Phase 2: Refinement (Sessions 20-100). Memory grows to 100+ entries. You discover which rules files get the most attention and which are ignored. You tighten security patterns after the

first false positive. You add project-specific skills for your validation workflows. The auto-build hook gets tuned for your specific build system (maybe you add a `--incremental` flag for faster Swift builds, or switch from full `tsc` to `tsc --noEmit` for TypeScript). You split MEMORY.md into an index file and topic-specific files when it hits 200 lines.

Phase 3: Institutional Knowledge (Sessions 100+). The stack becomes a knowledge base that rivals a project wiki. New engineers (human or AI) get the benefit of hundreds of sessions' worth of lessons learned. Rules files encode patterns that took months to discover. Memory contains pitfalls that would take a new team member weeks to encounter. Skills encode validation workflows that have been refined through dozens of iterations. The compound value exceeds the sum of individual entries.

At Phase 3, something interesting happens: the stack starts to prevent problems you did not anticipate. A new agent working on a feature you have never built before encounters a pitfall that is structurally similar to one in memory. The memory entry does not match exactly, but it provides enough context for the agent to avoid the same class of mistake. The institutional knowledge generalizes beyond its specific entries.

The Prompt Stack and Multi-Agent Systems

The prompt stack becomes even more valuable when you run multiple agents in parallel (as described in Post 6 with the worktree pipeline).

Problem: N agents, N independent contexts. Each agent starts fresh. Each one might make the same mistakes the others already made. Without shared context, parallel agents repeat each other's failures.

Solution: Shared stack, independent execution. All agents load the same Layers 1-3 and 7. They all know the project's build commands, common pitfalls, and institutional knowledge. They all have the same auto-build and security hooks. But each agent executes independently in its own worktree.

This means: - Agent A discovers a new pitfall. It gets added to memory. - Agent B, starting in the next pipeline run, reads that memory entry and avoids the pitfall. - Agent C, running QA on

Agent A's work, uses the same rules to evaluate code quality. - The merge queue, integrating all agents' work, benefits from the same dependency knowledge.

The stack is the shared context layer that makes parallel agents coherent. Without it, you have N independent agents making N independent mistakes. With it, you have N agents sharing one body of institutional knowledge.

Practical implications for multi-agent setups:

- Keep Layer 7 (memory) up to date between pipeline runs. Each run's discoveries should be merged into memory before the next run starts.
- Use Layer 3 (rules) to encode the QA criteria that all agents and QA reviewers share.
- Use Layer 4 (auto-build hook) to ensure every agent's work compiles independently. If it does not compile in the worktree, it will not compile after merge.
- Use Layer 5 (security hook) identically across all agents. A secret leaked by any agent is a secret in the repository.

What the Stack Does Not Solve

The prompt stack is not a silver bullet. It addresses structural problems – the kinds of failures that can be prevented by rules, hooks, and workflows. It does not address:

Incorrect reasoning. If the agent misunderstands the requirement, the stack cannot catch it. The auto-build hook verifies that code compiles, not that it does the right thing. The security hook checks for secrets, not for logic errors. Only the functional-validation skill (Layer 6) addresses correctness, and only when the agent follows it.

Novel problems. Memory contains known pitfalls. Rules contain known patterns. When the agent encounters a genuinely novel problem – one that no previous session has seen – the stack provides no guidance. The agent falls back to its own reasoning, which is exactly the mode where mistakes are most likely.

Social and organizational issues. The stack cannot prevent an agent from producing code that technically works but violates team norms not captured in rules files. If your team has

unwritten conventions about naming, error messages, or API design that nobody has documented in `.claude/rules/`, the agent will not follow them.

Performance optimization. The auto-build hook verifies compilation, not performance. An agent can produce code that compiles cleanly and passes all functional tests but introduces O(n^2) behavior, excessive memory allocation, or unnecessary network calls. Performance requires profiling tools, not prompt engineering.

These limitations are real but bounded. The stack eliminates the most common and most expensive failure modes. The remaining failures are the ones that require human judgment – and that is exactly where human review should focus.

The ROI Calculation

Is the prompt stack worth the setup time? Here is the math.

Setup cost: - Layer 1 (Global CLAUDE.md): 15 minutes to write - Layer 2 (Project CLAUDE.md): 30 minutes to customize from template - Layer 3 (Rules files): 45 minutes to fill in 9 templates - Layer 4 (Auto-build hook): 15 minutes to configure build commands - Layer 5 (Security hook): 10 minutes to review default patterns - Layer 6 (Skills): 30 minutes to customize validation workflows - Layer 7 (Memory): 0 minutes (grows organically) - **Total setup: ~2.5 hours**

Per-session savings (conservative estimates): - Auto-build hook prevents 1 cascading build failure per session: 15 minutes saved - Rules files prevent 2 “guessing” cycles per session: 10 minutes saved - Memory prevents 1 repeated mistake per session: 15 minutes saved - Security hook prevents 0.1 secret leaks per session: 30 minutes saved (amortized) - Skills prevent 1 “claimed done without evidence” per session: 20 minutes saved - **Total per-session savings: ~60-90 minutes**

Break-even: The stack pays for itself in 2-3 sessions. By session 10, you have saved 10+ hours. By session 100, you have saved 100+ hours. The return on investment is not 2x or 5x – it is 40-50x over the life of the project.

These numbers are conservative. They do not account for the catastrophic scenarios (leaked API key requiring key rotation and audit, broken production deployment due to untested code,

security vulnerability in committed credentials) that the stack prevents entirely. A single prevented incident can save days of remediation effort.

Frequently Asked Questions

Q: Does this work with AI tools other than Claude Code?

The architecture is tool-agnostic. The concepts – layered configuration, automatic build hooks, pre-commit security scans, validation skills, cross-session memory – apply to any AI coding tool that reads configuration files and supports hook mechanisms. The specific implementation (`.claude/` directory, `settings.local.json` format, skill invocation syntax) is Claude Code-specific. Porting to another tool requires adapting the hook configuration and file paths.

Q: Does the stack slow down the agent?

Yes, slightly. The auto-build hook adds 15-45 seconds per edit. The security hook adds 5-10 seconds per commit. Loading 9 rules files at session start adds a few seconds. The total overhead is roughly 30-60 seconds per meaningful edit cycle.

But this is a misleading comparison. Without the stack, agents are faster per edit but spend far more time debugging cascading failures, re-reading files they have already analyzed, and producing work that needs to be redone. The net effect is that sessions with the stack are shorter, not longer. The overhead per edit is more than offset by the elimination of debugging spirals.

Q: How do I share the stack with my team?

Layers 2-6 live in the repository and are shared via version control. Any team member who clones the repo gets the project CLAUDE.md, rules files, hooks, and skills automatically. Layer 1 (global CLAUDE.md) is personal – each team member customizes their own. Layer 7 (memory) lives in the repo and is shared, but team members should avoid conflicting edits to MEMORY.md.

Q: Can I use the stack with non-coding AI tasks?

The auto-build hook (Layer 4) and security hook (Layer 5) are coding-specific. But Layers 1-3 (configuration and rules), Layer 6 (skills), and Layer 7 (memory) apply to any AI task. A content writing project could have rules about tone and style, skills for review workflows, and memory about brand guidelines. The defense-in-depth architecture is not limited to code.

Q: What if my project already has CLAUDE.md but no hooks or rules?

Start by adding Layer 4 (auto-build hook). It is the highest-impact single addition. Then add Layer 5 (security hook). Then expand your CLAUDE.md into Layers 2-3 by moving domain knowledge into `.claude/rules/` files. The stack is designed to be adopted incrementally – you do not need all 7 layers from day one.

Where This Goes Next

The prompt engineering stack is version 1. It works well for individual developers and small teams working on single projects. Scaling to larger organizations and multi-project setups requires extensions that I am actively exploring.

Organization-wide Layer 1. Instead of each developer maintaining their own global CLAUDE.md, an organization could maintain a shared Layer 1 distributed via a package manager or dotfiles repository. Engineering standards, security policies, and model routing preferences would be consistent across all developers.

Dynamic Layer 4 configuration. Instead of hardcoding build commands in a bash script, Layer 4 could auto-detect the project's build system by reading `package.json`, `Cargo.toml`, `build.gradle`, or `Package.swift` at hook runtime. This would make the hook zero-configuration for projects that follow standard conventions.

Layer 5 integration with secret scanning services. The current security hook uses regex patterns. Integration with services like GitHub Secret Scanning, GitLeaks, or TruffleHog would provide more comprehensive detection with lower false positive rates.

Skill composition language. Currently, skills compose by convention (the agent is told to invoke both `functional-validation` and `ios-validation-runner`). A formal composition

syntax would allow skills to declare dependencies, define shared state, and chain verification steps across multiple skills.

Memory federation across projects. Pitfalls discovered in one project often apply to others. A shared memory layer across related projects (e.g., all iOS projects in an organization) would prevent the same pitfall from being discovered independently in each project.

The prompt engineering stack is not about writing better prompts. It is about building a system that makes it structurally impossible for an AI agent to cut corners, skip verification, leak secrets, or repeat past mistakes. Defense in depth. Every layer matters.

Companion repo with all templates, hooks, and skills: krzemienski/claude-prompt-stack

`#AgenticDevelopment` `#PromptEngineering` `#ClaudeCode` `#AIAgents` `#DefenseInDepth`

Part 7 of 11 in the Agentic Development series.

Diagrams & Visual Assets

SOCIAL MEDIA CARDS

Open `assets/twitter-card.html` and `assets/linkedin-card.html` in a browser to preview social cards.

Agentic Development: 10 Lessons from 8,481 AI Coding Sessions

Post 07 of 11